

PROVISIONAL APPLICATION FOR PATENT

Cover Sheet

(37 CFR 1.51(c))

TITLE OF THE INVENTION: Safe On-Device Neural Network Adaptation on Resource-Constrained Edge Hardware Without GPU or Cloud Dependency (referred to herein as “EdgeGuard”)

INVENTOR(S):

Given Name: Sara
Family Name: Aly
Residence (City, State, Country): Northridge, CA, United States

CORRESPONDENCE ADDRESS:

Name: Sara Aly
Address: 1709 E Glenoaks Blvd
City/State/ZIP: Glendale, CA 91206
Country: United States
Telephone: (424) 460-9975
Email: sarakhaleed.kaz@gmail.com

ENTITY STATUS: ☒ Micro Entity ☐ Small Entity ☐ Large Entity

(Check one. Most individual inventors qualify as micro entity — see filing instructions.)

FILING DATE: _____ *

SPECIFICATION PAGES: 52 pages (including drawings)

NUMBER OF CLAIMS: 20 (3 independent + 17 dependent)

NUMBER OF DRAWINGS: 12 figures

DOCKET NUMBER: (optional) _____

** Fill in before filing. See attached filing instructions.*

The attached specification, claims, abstract, and drawings constitute the complete provisional patent application.

PROVISIONAL PATENT APPLICATION
UNITED STATES PATENT AND TRADEMARK OFFICE

Safe On-Device Neural Network Adaptation
on Resource-Constrained Edge Hardware
Without GPU or Cloud Dependency
(“EdgeGuard”)

Inventor: Sara Aly

May 2026

Contents

1	FIELD OF THE INVENTION	7
2	BACKGROUND OF THE INVENTION	7
2.1	Limitations of GPU-Based Training for Edge Applications	7
2.2	Constraints of FPGA and SoC Platforms for Full-Model Training	8
2.3	Cloud Dependency and Its Limitations	9
2.4	The Gap: Edge Devices Can Infer But Cannot Learn	10
2.5	Limitations of Prior Art in On-Device Learning	10
2.5.1	Distinction from Dual-Bank Firmware Update Mechanisms	12
2.5.2	Distinction from Statistical Personalization in Wearable Biomedical Devices	12
2.6	Need for the Invention	13
3	SUMMARY OF THE INVENTION	13
4	DETAILED DESCRIPTION OF THE INVENTION	15
4.1	Sensor Subsystem and Data Acquisition	15
4.2	System Architecture Overview	16
4.3	Frozen-Backbone, Mutable-Head Architecture	18
4.3.1	Design Rationale	18
4.3.2	Parameter Partitioning	20
4.4	Gradient Engine: On-Device Backpropagation	21
4.4.1	Forward and Backward Pass	21
4.4.2	Optimization: SGD with Momentum	23
4.4.3	Numerical Safety Mechanisms	24
4.5	Dual-Slot Model Versioning with Flash Persistence	25
4.5.1	Stable/Candidate Architecture	25

4.5.2	NVS Flash Persistence	25
4.5.3	Factory Reset	26
4.6	Feedback-Driven Training Triggers	28
4.6.1	Confidence Drift Detection (Unsupervised)	28
4.6.2	User Correction Protocol (Supervised Learning)	29
4.6.3	Periodic Adaptation	29
4.7	Hardware-Enforced Resource Gating	30
4.8	Validation Engine: A/B Comparison with Safety Lockout	32
4.8.1	Validation Protocol	32
4.8.2	Promotion Criteria	33
4.8.3	Weight Divergence Detection	33
4.8.4	Safety Lockout	33
4.9	Memory Architecture	35
4.10	Measured Hardware Performance	38
4.11	Platform-Independent Applicability: SoC and FPGA Generalization	39
4.11.1	Architectural Abstraction	40
4.11.2	System-on-Chip (SoC) Applicability	41
4.11.3	FPGA Applicability	41
4.11.4	Why Full-Model Training Is Infeasible on These Platforms	42
4.11.5	Key Insight: Platform-Independent Innovation	43
5	CLAIMS	44
5.1	Independent Claims	44
5.2	Dependent Claims	48
6	ABSTRACT	51
7	DRAWINGS	51

1. FIELD OF THE INVENTION

The present invention relates generally to on-device machine learning on resource-constrained edge computing hardware, and more particularly to a system and method for performing feedback-driven, partial neural network training entirely on microcontroller, system-on-chip (SoC), or field-programmable gate array (FPGA) platforms — without requiring a graphics processing unit (GPU), cloud connectivity, external machine learning frameworks, or dynamic memory allocation. The invention is demonstrated in the domain of wearable biomedical monitoring but is applicable to any edge-deployed classification system that must adapt to evolving input distributions while operating autonomously.

2. BACKGROUND OF THE INVENTION

2.1 Limitations of GPU-Based Training for Edge Applications

Neural network training has historically required graphics processing units (GPUs) or dedicated accelerators. Modern training GPUs consume 150–350 W of power (e.g., NVIDIA A100: 300 W TDP per its product data sheet), cost \$1,000–\$40,000 per unit, require active cooling with heatsinks and fans, and occupy physical volumes incompatible with wearable or embedded form factors. Even mobile-class GPUs (e.g., NVIDIA Jetson series, per NVIDIA’s published module specifications) consume 10–30 W, require dedicated power supplies and thermal management, and cost \$200–\$500 — making them impractical for battery-powered, disposable, or cost-sensitive edge devices such as wearable health monitors, industrial IoT sensors, and remote environmental monitors.

As a consequence, the vast majority of edge-deployed machine learning systems perform **inference only**: a model is trained once on GPU/cloud infrastructure, its weights are frozen, and the fixed model is deployed to the edge device. Once deployed, the model cannot adapt to new conditions, drifting inputs, or individual users.

2.2 Constraints of FPGA and SoC Platforms for Full-Model Training

Field-programmable gate arrays (FPGAs) and embedded systems-on-chip (SoCs) offer low power and small form factors suitable for edge deployment. However, these platforms face fundamental limitations for neural network training:

- (1) **Memory constraints:** On-chip SRAM on typical edge SoCs ranges from 512 KB to 2 MB. Full backpropagation requires storing all intermediate activations (proportional to model depth $\times$ layer width $\times$ batch size), all gradients, and optimizer state (momentum, Adam second moments). For even modest models, this exceeds available on-chip memory by 10–100 $\times$.
- (2) **Limited or absent floating-point units:** Many microcontrollers (ARM Cortex-M0/M3, RISC-V RV32I) lack hardware floating-point support entirely. Those that include FPUs (Cortex-M4F, M7, Xtensa LX7) provide single-precision only, with no SIMD or matrix-multiply acceleration — making full-model gradient computation prohibitively slow.
- (3) **No framework support:** Standard machine learning frameworks (PyTorch, TensorFlow, JAX) target CUDA GPUs and x86/ARM Linux hosts. No production-grade backpropagation framework exists for bare-metal microcontroller firmware, FreeRTOS, or FPGA fabric. Existing TinyML toolchains (TensorFlow Lite Micro, CMSIS-NN, TVM) support inference only — they can *run* a pre-trained model but cannot *train* one.
- (4) **No dynamic memory allocation in safety-critical firmware:** Real-time and safety-critical embedded systems prohibit `malloc/free` during operation because dynamic allocation introduces non-deterministic latency, heap fragmentation, and out-of-memory failures. Standard training frameworks rely heavily on dynamic allocation for tensor creation, graph construction, and gradient accumulation.

These constraints collectively mean that while FPGAs and SoCs can efficiently *execute* neural network inference, they cannot *train* neural networks using conventional methods.

2.3 Cloud Dependency and Its Limitations

The conventional alternative to on-device training is cloud-based retraining: raw sensor data is transmitted to a server, training occurs on GPU infrastructure, and updated weights are sent back to the device. This approach introduces multiple critical limitations:

- (1) **Connectivity requirement:** Cloud retraining requires always-on network connectivity. This fails in scenarios where connectivity is unavailable, intermittent, or unreliable — including remote healthcare (rural clinics, developing regions), disaster response, military operations, underground or underwater environments, and aviation.
- (2) **Privacy and regulatory constraints:** Transmitting raw physiological data (ECG waveforms, heart rate, galvanic skin response) to cloud servers creates privacy risks and may violate healthcare regulations (HIPAA in the United States, GDPR in the European Union, and equivalent frameworks internationally). Many patients and healthcare providers require that sensitive biometric data remain on the device at all times.
- (3) **Latency:** Cloud round-trip times (100 ms to several seconds depending on network conditions) prevent real-time adaptation. A wearable device detecting accuracy degradation cannot wait for a cloud retraining cycle that may take minutes to hours.
- (4) **Recurring cost:** Cloud GPU compute (e.g., \$1–\$4/hour for GPU instances per published cloud pricing from major providers as of 2024–2025) creates ongoing operational costs that scale with the number of deployed devices and retraining frequency. For consumer wearables deployed at scale (millions of units), per-device cloud training is economically prohibitive.
- (5) **Single point of failure:** Cloud dependency means the device’s ability to adapt depends on server availability, API stability, and vendor continuity. Server outages, API deprecations, or vendor shutdowns permanently disable adaptation for all deployed devices.
- (6) **Vendor lock-in:** Cloud-based training ties the device manufacturer to a specific cloud provider’s infrastructure, pricing, and API contracts.

2.4 The Gap: Edge Devices Can Infer But Cannot Learn

The current state of edge machine learning creates a fundamental asymmetry: edge devices can *execute* neural networks with high efficiency but cannot *learn* or *adapt* once deployed. This gap has practical consequences for any edge classification system:

- (1) **Individual variation:** Population-trained models do not capture user-specific signal characteristics (e.g., unique gait patterns, resting heart rate baselines, skin impedance profiles). Performance degrades for users who deviate from training-set demographics.
- (2) **Sensor drift:** Physical sensors degrade over time — electrode impedance changes, optical coupling degrades, MEMS accelerometers shift calibration. A frozen model cannot compensate for these hardware changes.
- (3) **Distributional shift:** Environmental conditions (temperature, altitude, humidity) and behavioral changes alter input signal statistics in ways not represented in training data.
- (4) **Silent degradation:** Without adaptation, model accuracy degrades silently over weeks and months. The device continues producing predictions with high confidence even as those predictions become increasingly unreliable.

2.5 Limitations of Prior Art in On-Device Learning

Several prior systems have explored on-device neural network training, but none address the full set of requirements for autonomous, safety-critical edge deployment:

Table 1: Comparison of prior art on-device training systems with the present invention. ✓ = feature present, × = feature absent.

System	<i>No ext. framework</i>	<i>Zero malloc</i>	<i>Model versioning</i>	<i>A/B validation</i>	<i>Safety lockout</i>	<i>HW resource gates</i>
TinyTL (MIT, 2020)	×	×	×	×	×	×
TinyOL (2021)	×	×	×	×	×	×
TinyTrain (2023)	×	×	×	×	×	×
AlfES (Fraunhofer)	✓	×	×	×	×	×
Lin et al. (NeurIPS 2022)	×	×	×	×	×	×
Cloud Retraining	×	×	×	×	×	×
Present Invention	✓	✓	✓	✓	✓	✓

Key limitations of each prior system:

- **TinyTL**: Requires PyTorch; uses dynamic allocation; no safety mechanisms or versioning.
- **TinyOL**: Single output-layer update; relies on TensorFlow Lite Micro; no persistence.
- **TinyTrain**: Requires PyTorch backend; no rollback, validation, or resource gating.
- **AlfES**: General-purpose library using dynamic allocation; no safety or versioning.
- **Lin et al.**: Custom compiler for Cortex-M; no safety lockout, versioning, or monitoring.
- **Cloud Retraining**: Requires internet; recurring cost; privacy risks; single point of failure.

The critical gap across all prior art is the absence of a **complete safety and deployment framework**: no existing system combines zero-allocation training, dual-slot model versioning with atomic rollback, A/B validation before promotion, automatic safety lockout, hardware-enforced resource gating, and non-volatile flash persistence — all implemented without external libraries on bare-metal firmware. These safety properties are essential for any edge device where a degraded model could produce dangerous or clinically harmful results, and where the device must operate autonomously without human oversight or cloud connectivity.

2.5.1 Distinction from Dual-Bank Firmware Update Mechanisms

Dual-slot (“A/B”) storage with atomic promotion and rollback is a well-established pattern in embedded firmware update systems, including the Espressif ESP-IDF software development kit’s own over-the-air (OTA) update mechanism (`CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`), which maintains two firmware partitions and automatically reverts to the previous partition if a newly flashed firmware image fails to boot successfully. The present invention is readily distinguished from this and similar dual-bank firmware update mechanisms in two respects: (1) the present invention’s promotion criterion is a **multi-task classification accuracy comparison evaluated on a held-out labeled validation buffer**, not a binary boot-success signal; and (2) the dual slots in the present invention store **neural network classification head weights** produced by an on-device gradient-descent training process, not whole firmware images received from an external programming tool or OTA server. No prior dual-bank rollback mechanism known to the applicant evaluates the functional accuracy of the content being versioned before promotion; they instead rely on a proxy signal (successful boot) entirely unrelated to the quality of the update’s actual classification behavior.

2.5.2 Distinction from Statistical Personalization in Wearable Biomedical Devices

Personalization mechanisms exist in commercial wearable biomedical devices, including individualized calibration models for arrhythmia and cardiac monitoring (e.g., U.S. Patent No. 12,042,251, “Systems and methods of arrhythmia detection,” assigned to Braveheart Wireless, Inc.), which combine a population-trained prior model with individual sensor data through Bayesian or other statistical inference techniques to produce a personalized detection model. Such approaches differ fundamentally from the present invention in that they do not perform **gradient-based backpropagation through a neural network** on-device, and consequently have no analog to the present invention’s frozen-backbone/mutable-head partitioning, dual-slot model versioning, A/B accuracy validation gate, or safety lockout mechanisms — all of which are specific to the gradient-descent training context.

2.6 Need for the Invention

There exists a critical need for a **hardware-agnostic method for on-device neural network adaptation** that:

- (1) Operates entirely on the edge device with **no GPU, no cloud, no internet connectivity, and no external machine learning frameworks**
- (2) Runs on **sub-\$10 microcontrollers** (not just expensive GPU-equipped boards) with as little as 8 MB of external memory
- (3) Uses **only pre-allocated memory buffers** with zero dynamic allocation, suitable for safety-critical real-time firmware
- (4) **Guarantees through multiple independent safety mechanisms** that adaptation does not degrade model performance below the pre-deployment baseline
- (5) Is **platform-independent**: the same algorithmic architecture maps to microcontrollers (MCU), systems-on-chip (SoC), and FPGAs
- (6) Enables the device to **learn autonomously** — anywhere, anytime, with no external dependencies — adapting to individual users, sensor drift, and environmental changes throughout its operational lifetime

3. SUMMARY OF THE INVENTION

The present invention, referred to herein as “EdgeGuard,” solves the hardware problem of enabling neural network learning on resource-constrained edge devices that lack GPUs, have no internet connectivity, and cannot rely on external cloud infrastructure. The invention is designed from the ground up for the constraints of microcontroller, SoC, and FPGA platforms: limited memory (kilobytes to low megabytes), no dynamic memory allocation, no operating system in many configurations, limited or single-precision-only floating-point arithmetic, and strict real-time

deadlines. By freezing the compute-intensive CNN+SE+Transformer backbone and training only the lightweight classification heads, the system reduces the memory and compute requirements of on-device training to a level that fits within the resource envelope of a sub-\$10 microcontroller — eliminating the need for GPUs, cloud compute, or any external dependency.

Specifically, the present invention provides a complete system and method for on-device adaptive training of multi-task neural networks on microcontroller-class hardware (demonstrated on the Espressif ESP32-S3 system-on-chip with 8 MB PSRAM and 8 MB flash, total bill-of-materials under \$10). The invention enables feedback-driven, resource-constrained, partial parameter updates using only pre-allocated buffers, with no external machine learning libraries, no dynamic memory allocation, and no cloud connectivity. The device operates as a fully autonomous learning system — it can be deployed anywhere in the world, with no internet access, and will continuously adapt to its environment and user throughout its operational lifetime.

The key innovations of the invention include:

- (1) **Frozen-backbone, mutable-head architecture** for on-device training that freezes the convolutional feature extractor (CNN, squeeze-and-excitation block, and transformer encoder) and trains only the classification head parameters, reducing computational cost by approximately 95% while preventing catastrophic forgetting of learned feature representations.
- (2) **Hardware-enforced resource gating** that conditions training episodes on simultaneous satisfaction of five independent hardware constraints: available PSRAM, available heap memory, system-on-chip temperature, cooldown interval, and CPU load — ensuring training does not compromise real-time inference.
- (3) **Dual-slot model versioning with non-volatile flash persistence** that maintains stable and candidate weight sets in pre-allocated memory, enables atomic promotion and $O(1)$ rollback, and persists adapted weights across power cycles via NVS flash storage.
- (4) **A/B validation with safety lockdown** that evaluates candidate models against the stable model on a held-out validation buffer before promotion, and automatically disables adaptation after

a configurable number of consecutive failures.

- (5) **Multi-trigger feedback system** combining unsupervised confidence drift detection (exponential moving average), supervised user correction protocol, and periodic adaptation scheduling.
- (6) **Nine concentric safety mechanisms** (gradient clipping, weight magnitude clamping, NaN/Inf guards, validation gate, weight divergence scan, safety lockout, thermal guard, wall-clock budget, and factory reset) that collectively guarantee monotonic non-degradation of model performance.
- (7) **Zero-allocation training** where all gradient, momentum, activation cache, and model store buffers are pre-allocated during system initialization, guaranteeing deterministic real-time behavior with no heap fragmentation risk during training episodes.

The system has been demonstrated on real hardware processing five simultaneous biomedical sensor channels (ECG, PPG, AccX, AccY, AccZ) at 100 Hz, with measured inference latency of 2,338 ms per 10-second window (23.4% duty cycle), ~7.3 MB PSRAM remaining after all inference and training allocations, and only 8 KB additional flash overhead for the complete adaptive training subsystem.

4. DETAILED DESCRIPTION OF THE INVENTION

The following sections describe the reference implementation of EdgeGuard, demonstrated on the Espressif ESP32-S3 microcontroller in the domain of wearable biomedical monitoring.

4.1 Sensor Subsystem and Data Acquisition

The system acquires real-time biomedical and inertial data from a plurality of sensors connected to the microcontroller:

- **ECG (AD8232):** Single-lead electrocardiography analog front-end providing cardiac rhythm data for arrhythmia detection.
- **PPG (MAX30102):** Dual-wavelength (red/infrared) photoplethysmography sensor providing pulse waveform and blood oxygen proxy via I²C interface.
- **IMU (MPU6050):** 6-axis inertial measurement unit providing 3-axis accelerometer data (AccX, AccY, AccZ) for activity classification and motion context via I²C interface.

All sensors are sampled at a unified rate of 100 Hz. Data is organized into fixed-length windows of 1,000 samples (10 seconds) with a 5-channel input tensor representation (ECG, PPG, AccX, AccY, AccZ).

4.2 System Architecture Overview

The invention comprises seven tightly integrated software components executing on a microcontroller system-on-chip (demonstrated on ESP32-S3-N8R8 at 240 MHz with 8 MB PSRAM and 8 MB flash):

1. **Gradient Engine** — performs backpropagation through classification heads
2. **Model Store** — manages dual-slot weight versioning with flash persistence
3. **Feedback Controller** — detects adaptation triggers and buffers user corrections
4. **Resource Guard** — enforces hardware constraints before permitting training
5. **Validation Engine** — verifies candidate model safety before promotion
6. **Adaptive Trainer** — top-level orchestrator coordinating all components
7. **Training Configuration** — centralized governance of all hyperparameters

The system operates within the main firmware loop of a wearable biomedical monitoring device. After each inference window (10 seconds of multi-sensor data at 100 Hz), the system

evaluates whether adaptation should occur, verifies hardware resources are available, executes a bounded training episode if triggered, validates the result, and either promotes or rolls back the candidate model — all within a configurable wall-clock budget.

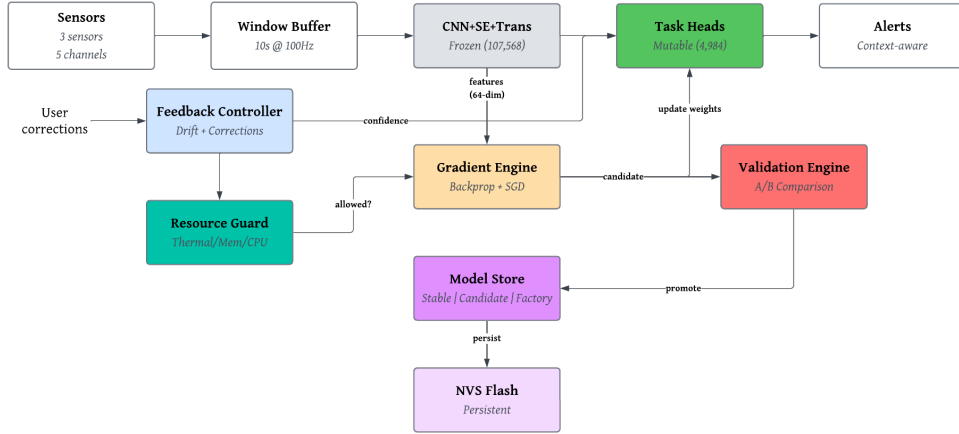


Figure 1: On-device adaptive training system architecture showing the frozen CNN+SE+Transformer backbone, mutable classification heads, and seven integrated components.

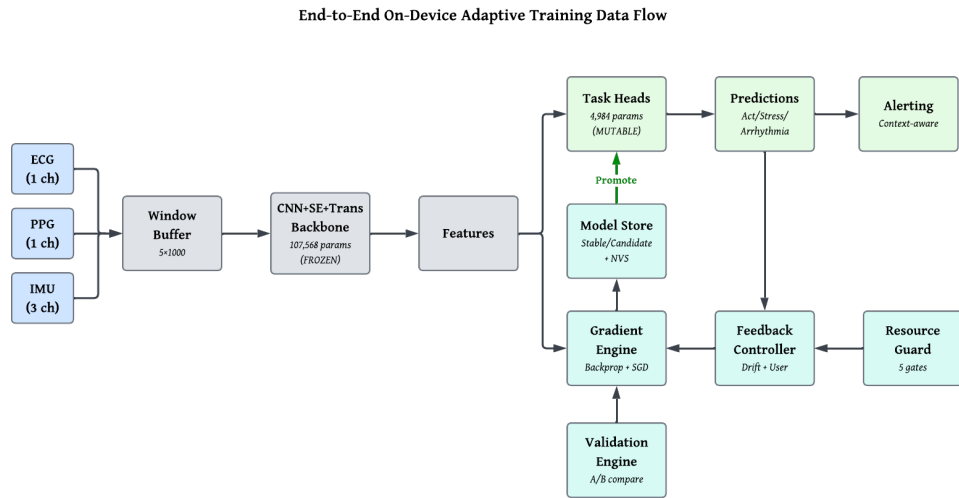


Figure 2: End-to-end data flow from sensor acquisition through CNN inference, feedback detection, gradient computation, validation, and model promotion.

On-Device Adaptation Cycle

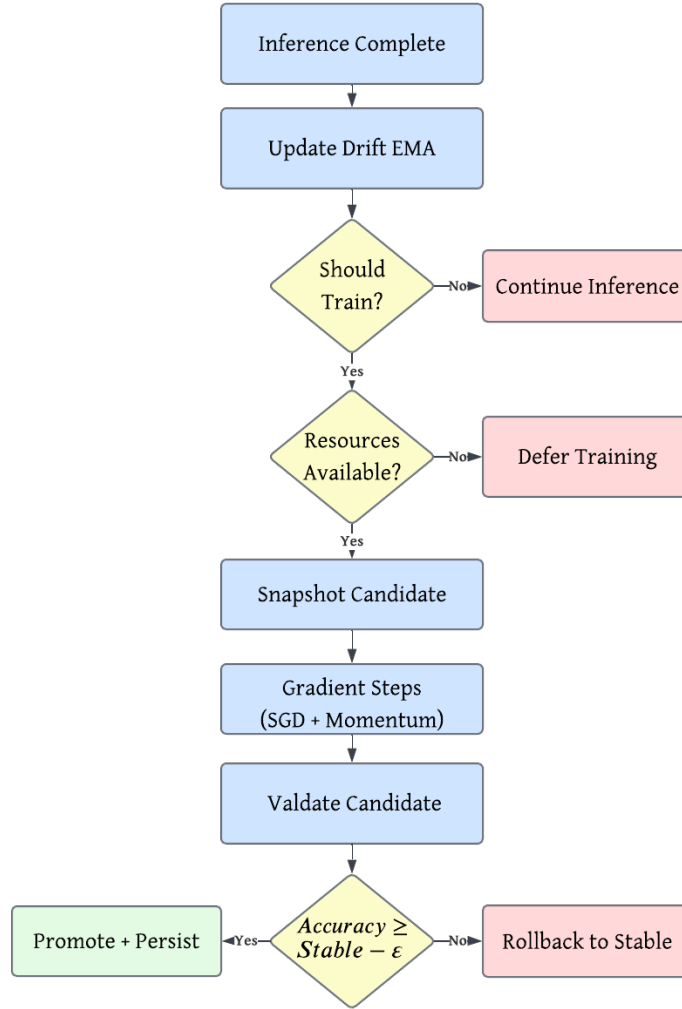


Figure 3: Decision flowchart for the complete on-device adaptation cycle showing the sequence from inference to trigger evaluation, resource gating, training, validation, and promotion/rollback.

4.3 Frozen-Backbone, Mutable-Head Architecture

4.3.1 Design Rationale

The neural network model comprises a frozen backbone — three 1D convolutional layers ($5 \rightarrow 32 \rightarrow 64 \rightarrow 64$) with squeeze-and-excitation (SE) attention, a linear projection with LayerNorm, and a two-layer transformer encoder (4 heads, 128-dim feed-forward) — totaling 107,568 parameters, plus three

task-specific classification heads (Activity with 4 classes, Stress with 2 classes, Arrhythmia with 2 classes, totaling 4,984 parameters across all heads). The grand total is 112,552 parameters occupying approximately 450 KB in 32-bit floating point.

The invention **freezes** the CNN+SE+Transformer backbone on-device and trains **only** the classification head parameters. This decision is motivated by three observations:

- (a) **Feature reuse:** Convolutional layers learn general signal representations (frequency components, morphological patterns, temporal correlations) that transfer well across users and conditions. These representations are expensive to learn and unlikely to improve from small amounts of on-device data.
- (b) **Compute reduction:** By excluding 107,568 backbone parameters (CNN, SE block, and transformer encoder) from gradient computation, the system reduces training cost by approximately 95% compared to full-model fine-tuning, while keeping the gradient path short (three fully-connected layers with intermediate LayerNorm) and numerically stable.
- (c) **Safety through architectural constraint:** Freezing the backbone prevents catastrophic forgetting of learned feature representations. Even if head adaptation fails completely, the system retains its full feature extraction capability and can roll back to the original classification heads.

4.3.2 Parameter Partitioning

Table 2: Parameter partitioning between frozen backbone and trainable classification heads. [†]FC1 within each head is frozen during on-device adaptation; only the LayerNorm, FC2, and FC3 layers of each head receive gradient updates.

Component	Layer	Parameters	Trainable
CNN Backbone	Conv1 ($5 \rightarrow 32$, $k=7$)	1,152	Frozen
	Conv2 ($32 \rightarrow 64$, $k=5$)	10,304	Frozen
	Conv3 ($64 \rightarrow 64$, $k=3$)	12,352	Frozen
SE Block	SE ($64 \rightarrow 16 \rightarrow 64$)	2,128	Frozen
Projection + LN	Linear ($64 \rightarrow 64$) + LayerNorm	4,288	Frozen
Transformer (per layer)	Self-Attention (4 heads)	16,640	Frozen
	Feed-Forward ($64 \rightarrow 128 \rightarrow 64$)	16,576	Frozen
	LayerNorm ($\times 2$ per layer)	256	Frozen
	(Subtotal per layer: 33,472; $\times 2$ layers = 66,944)		Frozen
Activity Head	FC1 ($64 \rightarrow 64$)	4,160	Frozen [†]
	LN + FC2 + FC3 ($64 \rightarrow 32 \rightarrow 4$)	2,340	✓
Stress Head	FC1 ($64 \rightarrow 48$)	3,120	Frozen [†]
	LN + FC2 + FC3 ($48 \rightarrow 24 \rightarrow 2$)	1,322	✓
Arrhythmia Head	FC1 ($64 \rightarrow 48$)	3,120	Frozen [†]
	LN + FC2 + FC3 ($48 \rightarrow 24 \rightarrow 2$)	1,322	✓
Total frozen		107,568	
Total trainable		4,984	
Grand total		112,552	

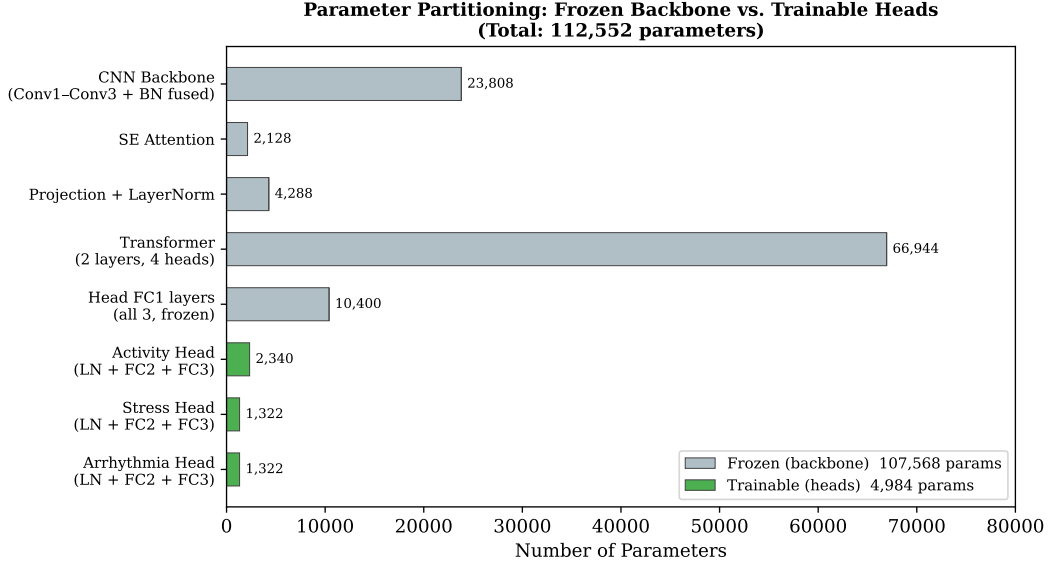


Figure 4: Parameter partitioning between the frozen backbone (107,568 parameters: CNN + SE + Transformer) and the trainable classification heads (4,984 parameters) across three task-specific heads.

4.4 Gradient Engine: On-Device Backpropagation

The Gradient Engine is the component responsible for computing how model weights should be updated to reduce prediction errors. It implements a standard neural network training procedure called *backpropagation* (backward propagation of errors), but operates entirely within pre-allocated memory buffers on the microcontroller. The engine processes one labeled training sample at a time: it runs the sample through the classification head (forward pass), computes the error between the prediction and the true label, then propagates this error backward through each layer to determine how much each weight contributed to the error (backward pass). The resulting gradients are used to adjust the weights via stochastic gradient descent.

4.4.1 Forward and Backward Pass

The Gradient Engine implements backpropagation through the three-layer classification heads (with intermediate LayerNorm) using the cross-entropy loss function. Given a classification head

with output layer logits $z \in \mathbb{R}^K$ (where K is the number of classes for that task), the softmax function produces a probability distribution $\hat{y} = \text{softmax}(z)$ where $\hat{y}_i = e^{z_i} / \sum_{j=1}^K e^{z_j}$. For ground-truth class $c \in \{1, \dots, K\}$, the loss and gradients are computed as follows:

$$\text{Cross-entropy loss: } \mathcal{L} = -\log \hat{y}_c \quad (1)$$

$$\text{Logit gradient: } \frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - \mathbf{1}[i = c] \quad (2)$$

$$\text{FC3 weight gradient: } \nabla_{W_3} = \delta_z \cdot \mathbf{h}_2^\top \quad (3)$$

$$\text{Backprop through ReLU (layer 2): } \delta_{h_2} = W_3^\top \delta_z \odot \mathbf{1}[\mathbf{h}_2 > 0] \quad (4)$$

$$\text{FC2 weight gradient: } \nabla_{W_2} = \delta_{h_2} \cdot \hat{\mathbf{h}}_1^\top \quad (5)$$

$$\text{Backprop through LayerNorm + ReLU: } \delta_{h_1} = W_2^\top \delta_{h_2} \odot \nabla_{\text{LN}} \odot \mathbf{1}[\mathbf{h}_1 > 0] \quad (6)$$

$$\text{FC1 weight gradient: } \nabla_{W_1} = \delta_{h_1} \cdot \mathbf{x}^\top \quad (7)$$

where the symbols are defined as:

- $\mathcal{L}$: cross-entropy loss (scalar)
- $\hat{y}_c$: predicted probability assigned to the true class c
- z_i : the i -th logit (raw, unnormalized output of the final fully-connected layer FC3 before softmax)
- $\mathbf{1}[i = c]$: indicator function equal to 1 when $i = c$ and 0 otherwise
- $\delta_z = \frac{\partial \mathcal{L}}{\partial z}$: the **logit gradient**, i.e., the gradient of the loss with respect to the pre-softmax logits. This quantity has a convenient closed-form: $\hat{y}_i - \mathbf{1}[i = c]$, meaning the gradient is simply the predicted probability minus the one-hot encoding of the true label
- $W_3 \in \mathbb{R}^{K \times d_2}$: weight matrix of the third fully-connected layer (FC3), where d_2 is the second hidden dimension
- $\mathbf{h}_2 \in \mathbb{R}^{d_2}$: hidden activation vector output by FC2 after ReLU activation
- $W_2 \in \mathbb{R}^{d_2 \times d_1}$: weight matrix of the second fully-connected layer (FC2)
- $\hat{\mathbf{h}}_1 \in \mathbb{R}^{d_1}$: LayerNorm-normalized output of FC1 hidden activations

- ∇_{LN} : gradient through the LayerNorm layer (applied between FC1 and FC2)
- $\delta_{h_1}, \delta_{h_2}$: gradients with respect to the hidden activations at each layer, obtained by backpropagating through ReLU derivatives ($\mathbf{1}[\mathbf{h} > 0]$, which passes gradients only where activations were positive)
- $\odot$: element-wise (Hadamard) multiplication
- $W_1 \in \mathbb{R}^{d_1 \times 64}$: weight matrix of the first fully-connected layer (FC1), where d_1 is the first hidden dimension
- $\mathbf{x} \in \mathbb{R}^{64}$: feature vector from the frozen backbone (input to the classification head)
- $\nabla_{W_1}, \nabla_{W_2}, \nabla_{W_3}$: gradient matrices for FC1, FC2, and FC3 weights respectively

All operations use pre-allocated float32 buffers in PSRAM. No dynamic memory allocation occurs during training.

4.4.2 Optimization: SGD with Momentum

Parameter updates use stochastic gradient descent with momentum:

$$\mathbf{v}(t) = \mu \cdot \mathbf{v}(t-1) + \nabla \theta \quad (8)$$

$$\theta(t) = \theta(t-1) - \eta \cdot \mathbf{v}(t) \quad (9)$$

where the symbols are:

- $\mathbf{v}(t)$: momentum velocity vector at training step t , accumulated across steps to smooth the gradient signal
- $\mu = 0.9$: momentum coefficient controlling how much of the previous velocity carries forward
- $\nabla \theta$: gradient of the loss with respect to all trainable parameters (computed by the backward pass in Section 4.4.1)
- $\theta(t)$: the trainable parameter vector (all mutable head weights and biases) at step t
- η : learning rate, starting at 0.05 with per-episode decay factor 0.95 down to a floor of 0.005.

The relatively aggressive initial rate (compared to typical large-batch cloud training) is appropriate for the few-shot, single-sample SGD regime of on-device adaptation, where only a small number of user corrections are available per training episode.

4.4.3 Numerical Safety Mechanisms

Three mechanisms prevent numerical instability:

- (a) **Gradient clipping:** Before applying gradients, the L_2 norm of the complete gradient vector is computed. If it exceeds a threshold (default: 1.0), all gradients are uniformly scaled to the threshold norm.
- (b) **Weight magnitude clamping:** After each parameter update, every weight is clamped to the range $[-10.0, +10.0]$.
- (c) **NaN/Inf detection:** Every forward pass output, backward pass gradient, and post-update weight value is explicitly scanned for NaN and Inf values. If any are detected, the entire training step is aborted and the weights are rolled back.

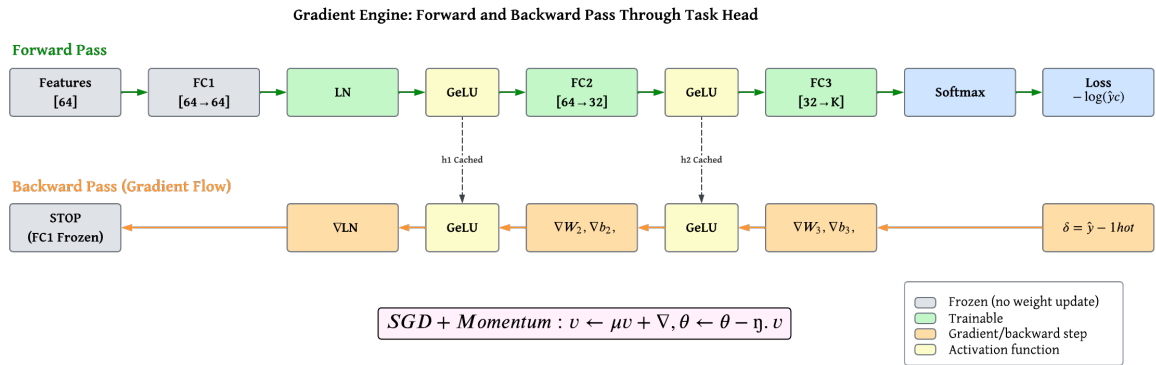


Figure 5: Gradient engine dataflow showing forward pass, backward pass with cached activations, gradient clipping, and SGD with momentum update. All operations use pre-allocated float32 buffers in PSRAM.

4.5 Dual-Slot Model Versioning with Flash Persistence

4.5.1 Stable/Candidate Architecture

The Model Store maintains two complete weight sets in pre-allocated PSRAM buffers:

- **Stable slot (~21 KB, 20,768 B):** The currently deployed, validated weights used for real-time inference.
- **Candidate slot (~21 KB, 20,768 B):** Weights being adapted by the Gradient Engine, not yet validated.

This dual-slot architecture enables three key properties:

- (1) **Atomic promotion:** Candidate weights replace the stable slot in a single buffer copy, with no intermediate state where inference uses partially updated weights.
- (2) $O(1)$ **rollback:** Stable weights are restored via a single `memcpy`, discarding all candidate modifications instantly.
- (3) **Non-blocking inference:** Real-time inference always uses the stable slot and is not affected by ongoing training or validation.

4.5.2 NVS Flash Persistence

After each successful promotion, the new stable weights are serialized and persisted to the ESP32 Non-Volatile Storage (NVS) flash partition. The persistence format consists of:

- A single binary blob containing all three task heads (~21 KB, 20,768 B)
- A generation counter incremented on each promotion
- Namespace isolation preventing interference with other NVS consumers

4.5.3 Factory Reset

A factory reset operation provides a guaranteed recovery path to the original pre-deployment model. The operation performs three actions in sequence:

- (1) **Weight restoration:** The original model weights, which are compiled into the firmware binary as read-only `.inc` array files during the initial build process, are copied from flash ROM into the active weight buffers in PSRAM. These ROM weights represent the population-trained baseline model and are always available regardless of the state of the adapted weights.
- (2) **NVS clearing:** All adapted weights stored in the ESP32 Non-Volatile Storage (NVS) flash partition are erased. This ensures that the next boot cycle loads the factory weights rather than any previously adapted weights.
- (3) **Generation counter reset:** The *generation counter* — a monotonically increasing integer that is incremented by one each time a candidate model is successfully promoted to stable — is reset to zero. This counter serves as an auditable record of how many successful adaptation cycles have occurred since the last factory reset, providing a measure of the model's cumulative drift from the factory baseline.

Factory reset can be triggered via the serial command `RESET` or programmatically when the system detects irrecoverable state (e.g., corrupted NVS data).

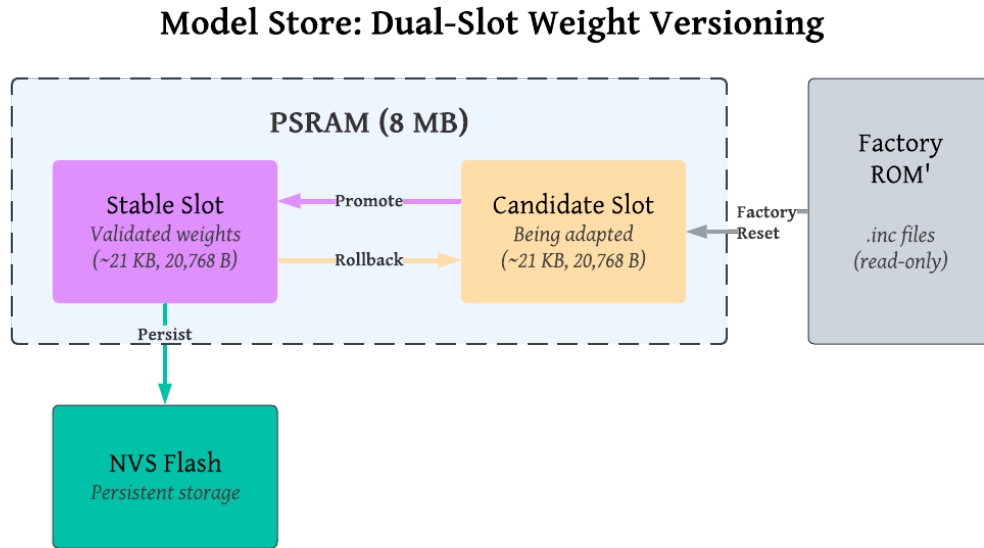


Figure 6: Dual-slot model versioning architecture showing stable slot, candidate slot, promotion path, rollback path, NVS flash persistence, and factory reset.

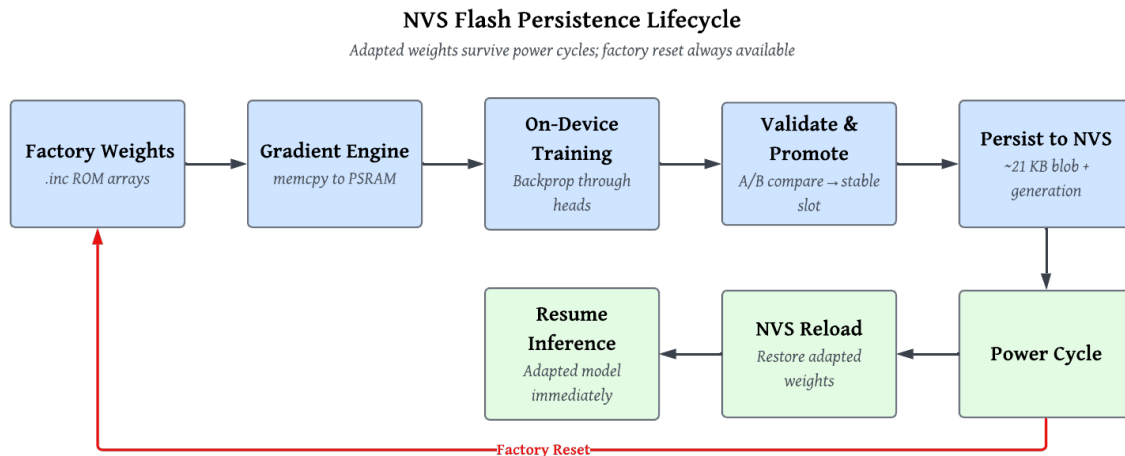


Figure 7: NVS flash persistence lifecycle showing factory weights loaded to PSRAM, on-device training, validation and promotion, persistence to NVS flash, and weight reload after power cycling. Factory reset always available.

4.6 Feedback-Driven Training Triggers

The Feedback Controller determines *when* the system should adapt. Three independent trigger mechanisms are implemented:

4.6.1 Confidence Drift Detection (Unsupervised)

An **Exponential Moving Average (EMA)** of prediction confidence is tracked independently for each task head. The EMA is a weighted running average that gives more weight to recent observations while smoothing out short-term noise. It is computed recursively at each inference window t as:

$$\text{EMA}_t = \alpha \cdot c_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \quad (10)$$

where:

- c_t : the *confidence* at window t , defined as the maximum softmax output probability $\max_i \hat{y}_i$. A high value (close to 1.0) indicates the model is confident in its prediction; a low value indicates uncertainty.
- $\alpha = 0.1$: the smoothing factor, which controls how quickly the EMA responds to new observations. A small α produces a smoother average that is less sensitive to individual outliers, while a larger α makes the EMA more responsive. The value 0.1 means the effective averaging window spans approximately $1/\alpha = 10$ recent inference windows (~ 100 seconds at one window per 10 seconds).
- EMA_{t-1} : the EMA value from the previous inference window.

Threshold selection: When the EMA drops below a threshold $\theta = 0.45$ for three or more consecutive windows, distributional drift is detected and adaptation is triggered. The threshold value of 0.45 was chosen empirically based on the following reasoning: a softmax confidence of 0.45 for a multi-class task (e.g., the 4-class activity task, where random chance is 25%) still indicates above-chance performance, but represents a significant decline from typical operating confidence (0.7–0.9 for well-calibrated predictions). The consecutive-window requirement (3 windows = 30

seconds) prevents adaptation from being triggered by transient noise or brief signal artifacts, while remaining responsive enough to detect sustained distributional shifts within minutes.

4.6.2 User Correction Protocol (Supervised Learning)

A serial command interface allows a clinician or user to provide ground-truth labels for the most recent prediction. This is a **supervised learning** mechanism — not reinforcement learning — because the user provides explicit ground-truth class labels (e.g., “the correct activity is Walking”) rather than scalar reward signals. Each correction creates a standard supervised training pair: the 64-dimensional feature vector from the corresponding inference window (input) paired with the user-provided class label (target). These labeled pairs are stored in a 32-entry ring buffer and used as training samples for cross-entropy loss minimization via backpropagation, exactly as in conventional supervised learning. The key distinction from reinforcement learning is that the system receives direct categorical labels, not delayed reward signals, and optimization uses gradient descent on a classification loss rather than policy gradient or value-function methods.

4.6.3 Periodic Adaptation

Accumulated corrections are consumed on a configurable schedule (default: every 60 inference windows, approximately 10 minutes).

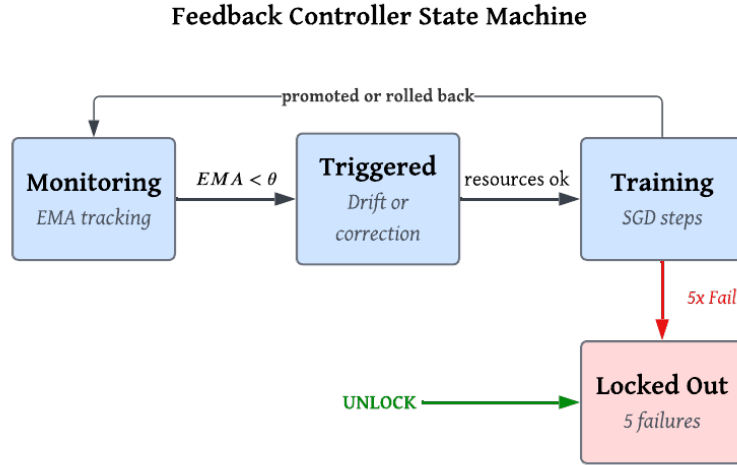


Figure 8: Feedback controller state machine with four states (Monitoring, Triggered, Training, Locked Out) and three independent trigger paths: confidence drift, user correction, and periodic timer. The system enters the Locked Out state after five consecutive validation failures and is cleared via the UNLOCK command or a factory reset.

4.7 Hardware-Enforced Resource Gating

The Resource Guard enforces hardware constraints before allowing any training episode. **All five checks must pass simultaneously:**

Table 3: Resource gating criteria. All conditions must be satisfied before training is permitted.

Resource	Condition	Threshold
PSRAM	Free bytes $\geq$ minimum	2 MB
Heap	Free bytes $\geq$ minimum	50 KB
Temperature	SoC temp $<$ maximum	65°C*
Cooldown	Time since last train $\geq$ minimum	30 s
CPU Load	Last inference latency $\leq$ maximum	3,000 ms

*The ESP32-S3 has a rated operating temperature range of -40°C to $+85^{\circ}\text{C}$ per the Espressif ESP32-S3 datasheet; the 65°C threshold provides a 20°C safety margin below the maximum rated junction temperature.

The temperature check uses an **API-adaptive implementation** that selects between different hardware abstraction layer (HAL) interfaces at compile time based on the SDK version. Specifically, the firmware uses C preprocessor `_has_include()` directives to detect whether the ESP-IDF temperature sensor API is available (`driver/temperature_sensor.h` in ESP-IDF 5.x) or whether the legacy API (`driver/temp_sensor.h` in ESP-IDF 4.4) should be used instead. If neither API is available, the temperature gate returns a safe default (below threshold), ensuring the system compiles and operates correctly across SDK versions without manual configuration. This compile-time selection enables the same firmware source to target multiple ESP-IDF versions while maintaining the temperature-based resource gating functionality.

Additionally, each training episode has a **wall-clock budget** of 2,000 ms (30 SGD steps at approximately 5 ms each). If exceeded, training stops with whatever steps have completed.

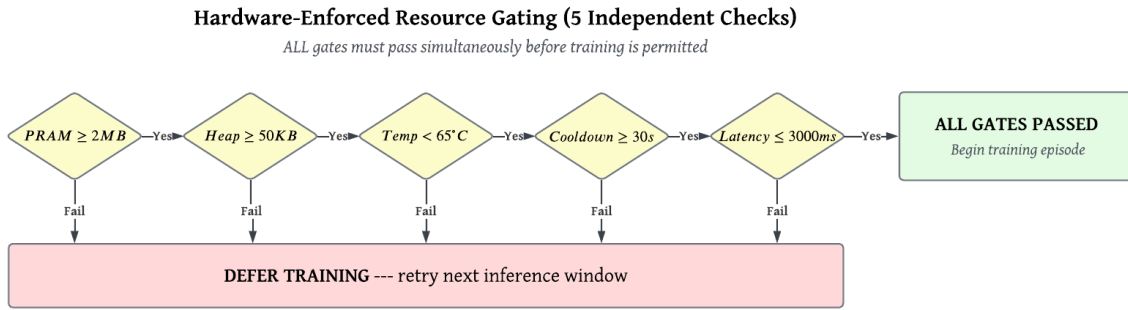


Figure 9: Hardware-enforced resource gating decision tree. All five independent checks (PSRAM, heap, temperature, cooldown, CPU load) must pass simultaneously before training is permitted. Failure of any single gate defers training to the next inference window.

4.8 Validation Engine: A/B Comparison with Safety Lockout

4.8.1 Validation Protocol

After each training episode, candidate and stable models are evaluated on a held-out validation buffer (up to 16 recent labeled samples):

1. **Snapshot** candidate weights into the Model Store’s candidate slot
2. **Evaluate candidate accuracy** using adapted weights
3. **Rollback** to stable weights from the stable slot
4. **Evaluate stable accuracy** on the same data
5. **Restore candidate** weights from the candidate slot

This protocol requires no dynamic memory allocation.

4.8.2 Promotion Criteria

The candidate is promoted if and only if:

$$\text{acc}_{\text{candidate}}(t) \geq \text{acc}_{\text{stable}}(t) - \epsilon \quad \forall t \in \{0, 1, 2\} \quad (11)$$

where $\text{acc}_{\text{candidate}}(t)$ and $\text{acc}_{\text{stable}}(t)$ are the classification accuracies of the candidate and stable models respectively on validation data for task t , ϵ is a configurable tolerance margin, and $t \in \{0, 1, 2\}$ indexes the three tasks (Activity, Stress, Arrhythmia). The candidate must meet this criterion for *all* tasks simultaneously — a single task falling below the threshold causes a rollback. In the validated hardware embodiment, $\epsilon = 1.0$ (one percentage point), a deliberately wide margin chosen because the few-shot, single-correction-buffer validation set (up to 16 samples) produces high-variance accuracy estimates; a tighter margin (e.g., $\epsilon = 0.05$, intended for production deployment with larger validation buffers) would reject otherwise-improving candidates due to sampling noise alone. Hardware validation confirmed the gate functions correctly at this setting: six training episodes with small correction sets (1–5 samples) were all correctly rolled back, demonstrating that the gate prevents premature promotion from insufficient data even with a wide tolerance margin.

4.8.3 Weight Divergence Detection

Before accuracy evaluation, all candidate weights are scanned for values exceeding a divergence threshold ($|w| > 50.0$), providing an $O(n)$ early-exit for numerically unstable candidates.

4.8.4 Safety Lockout

If a configurable number of consecutive candidates fail validation (five, in the validated hardware embodiment), the system enters a safety lockout state that disables all adaptation. The five-failure threshold is a practical compromise: occasional failures are expected during normal exploration with small validation sets, but five consecutive failures indicate a systematic problem (e.g., corrupted corrections or fundamental distributional mismatch) rather than benign sampling variance.

Hardware testing confirmed this mechanism activates correctly after five rollbacks and can be successfully cleared via the UNLOCK serial command or a factory reset.

Table 4 summarizes all nine concentric safety mechanisms in execution order.

Table 4: Nine concentric safety mechanisms in the on-device adaptive training system, listed in execution order. Each mechanism provides an independent defense layer. Collectively, these mechanisms are designed to prevent adaptation from degrading model performance below the pre-deployment baseline.

#	Mechanism	When	Condition	Action
1	Resource Guard	Before training	$\text{PSRAM} \geq 2 \text{ MB}$ $\text{Heap} \geq 50 \text{ KB}$ $\text{Temp} < 65^\circ\text{C}$ $\text{Cooldown} \geq 30 \text{ s}$ $\text{Latency} \leq 3,000 \text{ ms}$	Defer
2	Gradient Clipping	Backprop	$\ \nabla\ _2 > 1.0$	Scale to 1.0
3	Weight Clamping	SGD step	$ w_i > 10.0$	Clamp ± 10.0
4	NaN/Inf Guards	Every op	NaN or Inf	Abort, rollback
5	Divergence Scan	Pre-valid.	Any $ w > 50.0$	Reject candidate
6	A/B Validation	Post-train	$\text{acc}_c < \text{acc}_s - \epsilon$	Rollback
7	Safety Lockout	Post-reject	5 consec. fails	Disable training
8	Wall-Clock Budget	Mid-episode	$> 2,000 \text{ ms}$	Stop early
9	Factory Reset	On command	Manual or corrupt	Restore ROM weights

A/B Validation Protocol (Zero-Allocation)

All weight swaps use pre-allocated PSRAM slots -- no malloc/free

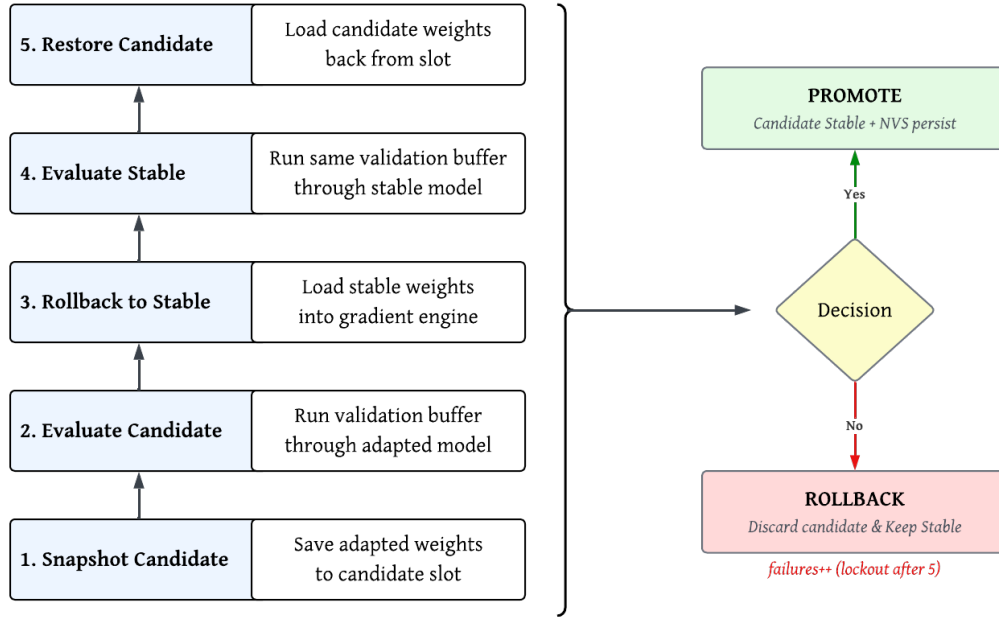


Figure 10: A/B validation protocol (zero-allocation). The 5-step procedure evaluates candidate and stable models on the same validation buffer using pre-allocated weight slots, then promotes or rolls back based on multi-task accuracy comparison.

4.9 Memory Architecture

A key design principle of the invention is that **all memory for both inference and training is pre-allocated during system initialization**, with zero dynamic allocation (malloc/free) during operation. This section details the complete memory budget, distinguishing between three categories of buffers:

- (1) **Weight buffers:** Contiguous arrays in PSRAM that store the float32 model parameters. The *mutable weights* buffer holds the active copy being used and modified by the gradient engine. The *stable slot* and *candidate slot* in the Model Store are separate copies used for versioning and rollback (Section 4.5).

- (2) **Gradient and momentum buffers:** Arrays of the same size as the mutable weights, used to store computed gradients (∇_W, ∇_b) and the momentum velocity vectors ($\mathbf{v}$) for SGD with momentum. These are overwritten each training step.
- (3) **Activation caches:** Small buffers (272 bytes per head $\times$ 3 heads = 816 bytes total) that store the intermediate hidden-layer activations ($\mathbf{h}$) from the forward pass, which are needed during the backward pass for gradient computation.
- (4) **Sample buffers:** The *correction buffer* (32 entries in internal SRAM) stores recent (feature vector, label) pairs provided by user corrections. The *validation buffer* (16 entries in SRAM) stores the most recent labeled samples used for A/B accuracy comparison.

Table 4.9 provides the complete byte-level memory budget.

Table 5: Complete memory budget for on-device adaptive training on ESP32-S3 (8 MB PSRAM).

Buffer	Size (bytes)	Location
<i>Inference (existing)</i>		
Input normalized [5×1000]	20,000	PSRAM
Conv1 output [32×1000]	128,000	PSRAM
Conv2 output [64×500]	128,000	PSRAM
Conv3 output [64×250]	64,000	PSRAM
SE + Transformer + projected + heads	$\sim 324,000$	PSRAM
<i>Training (new)</i>		
Mutable weights (3 heads)	20,768	PSRAM
Momentum buffers (3 heads)	20,768	PSRAM
Gradient buffers (3 heads)	20,768	PSRAM
Activation caches (3 heads)	952	PSRAM
Model Store: stable slot	20,768	PSRAM
Model Store: candidate slot	20,768	PSRAM
Correction buffer (32 samples)	8,448	SRAM
Validation buffer (16 samples)	4,672	SRAM
Total inference	$\sim 703,844$	
Total training	$\sim 117,912$	
Grand total	$\sim 821,756$	
Remaining PSRAM	~ 7.3 MB	

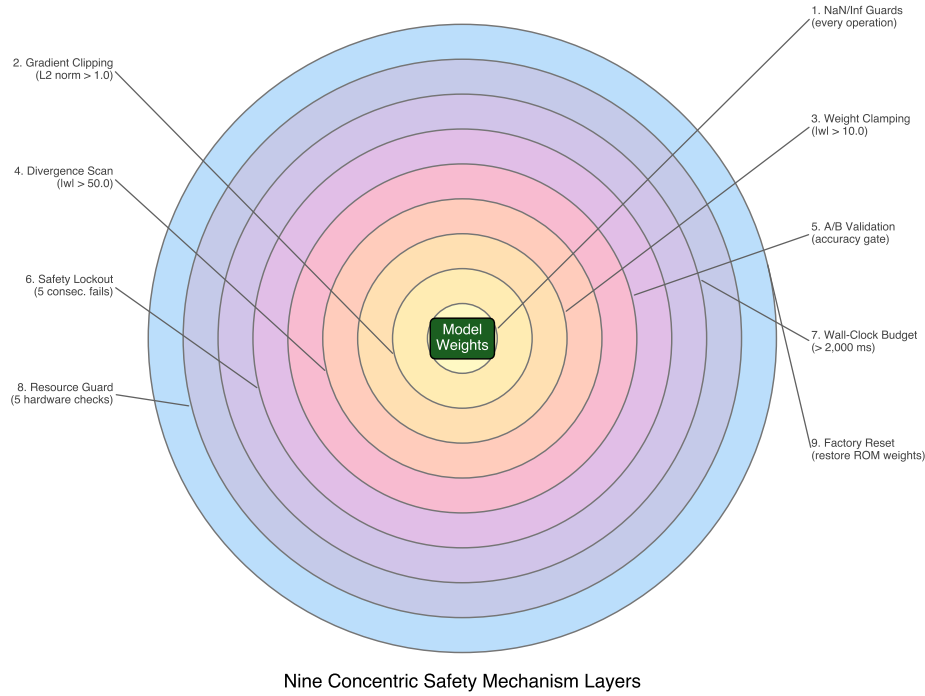


Figure 11: Nine concentric safety layers surrounding the deployed model, from innermost (NaN/Inf guards) to outermost (factory reset), collectively guaranteeing monotonic non-degradation.

4.10 Measured Hardware Performance

The adaptive training system has been validated on real ESP32-S3-N8R8 hardware with all three sensors (ECG, PPG, IMU) active and the complete training subsystem enabled. Table 4.10 presents runtime measurements from consecutive inference windows and training episodes, demonstrating that the training system fits within the resource envelope of the microcontroller with substantial headroom. The 2,338 ms inference latency per 10-second window represents a 23.4% duty cycle, leaving 76.6% of each window available for sensor acquisition, adaptive training episodes, and idle sleep. The PSRAM minimum watermark of ~ 7.2 MB (out of 8 MB total) confirms that the combined inference and training allocations leave ample headroom above the resource guard's 2 MB minimum threshold.

Table 6: Measured runtime performance on ESP32-S3-N8R8 with adaptive training enabled, averaged over 70+ consecutive inference windows.

Metric	Measured Value
CNN+SE+Transformer + heads inference compute	2,334 ms (± 2 ms)
Post-processing (softmax + alerting)	4 ms
Total per-window processing	2,338 ms
Compute duty cycle	23.4%
Training episode duration (6 episodes)	10.4–23.7 ms (mean 16.0 ms)
Training cross-entropy loss range	0.49–1.71
Heap free / min watermark	~ 293 KB / ~ 288 KB
PSRAM free / min watermark	~ 7.3 MB / ~ 7.2 MB
Firmware flash (with training)	767,936 B (23.0%)
Training system overhead	+8,236 B (+0.2%)

The complete training pipeline was validated end-to-end by sending user corrections via the serial command interface and forcing training episodes on live sensor data. Six training episodes were executed; all candidates were correctly rolled back by the A/B validation gate (the small number of corrections was insufficient to produce a candidate that outperformed the stable model across all three task heads). After five consecutive rollbacks, the safety lockout engaged as designed. The UNLOCK serial command successfully cleared the lockout, and the RESET command restored factory weights and reset the generation counter to zero, confirming NVS flash persistence and factory reset functionality.

4.11 Platform-Independent Applicability: SoC and FPGA Generalization

The algorithmic architecture described herein — frozen backbone, mutable classification heads, gradient engine, dual-slot model store, validation engine, and resource guard — is defined at the **algorithmic level** and is not intrinsically tied to any specific hardware platform. While the refer-

ence implementation demonstrates the system on an ESP32-S3 microcontroller (MCU), the same architectural principles map directly to System-on-Chip (SoC) and Field-Programmable Gate Array (FPGA) targets.

4.11.1 Architectural Abstraction

The invention defines a **Hardware Abstraction Layer (HAL)** — a software interface that decouples the adaptive training algorithms from the specific hardware they run on. The HAL provides a set of standardized function calls (e.g., “read temperature,” “allocate weight buffer,” “persist to flash”) that each platform implements differently. This allows the same training logic to run unmodified on an MCU, SoC, or FPGA by simply swapping the HAL implementation. The six core components of the adaptive training system (Gradient Engine, Model Store, Feedback Controller, Resource Guard, Validation Engine, and Adaptive Trainer) interact with hardware only through the following six HAL abstractions:

- (1) **Weight storage:** Any addressable memory (PSRAM on MCU, on-chip SRAM or DDR on SoC, Block RAM or HBM on FPGA)
- (2) **Gradient computation:** Any arithmetic unit (sequential C++ on MCU, CMSIS-NN or NPU acceleration on SoC, RTL datapath pipelines on FPGA)
- (3) **Non-volatile persistence:** Any persistent storage (NVS flash on MCU, eMMC or QSPI flash on SoC, SD card or SPI flash on FPGA)
- (4) **Temperature monitoring:** Any thermal sensor (internal ADC on MCU, PMU thermal unit on SoC, XADC or on-die sensor on FPGA)
- (5) **Resource monitoring:** Platform-specific free memory and CPU load queries, abstracted behind a common interface
- (6) **Timer/clock:** Wall-clock timing for training budgets and cooldown intervals (hardware timers on MCU, system tick on SoC, clock counter on FPGA)

4.11.2 System-on-Chip (SoC) Applicability

On ARM Cortex-M7/A series SoCs (e.g., NXP i.MX RT, STM32H7, or Qualcomm QCC series), the adaptive training architecture maps as follows:

Table 7: Mapping of adaptive training components to SoC hardware resources.

Component	SoC Implementation
Frozen backbone	ARM cores + DSP/NPU co-processor
Mutable heads	On-chip SRAM or DDR-backed buffers
Gradient engine	CMSIS-NN optimized matrix operations
Weight storage	On-chip SRAM or DMA-mapped DDR
Gradient buffers	DMA-mapped double-buffered regions
NVS persistence	eMMC partition or QSPI flash sector
Temperature sensor	SoC PMU thermal management unit

SoC implementations benefit from hardware DSP/NPU acceleration for gradient computation, DMA-based buffer management for zero-copy weight transfers, and integrated power management units for thermal gating — all of which are compatible with the resource gating and safety mechanisms described herein.

4.11.3 FPGA Applicability

On FPGA platforms (e.g., Xilinx Zynq, Intel Cyclone, Lattice ECP5), the architecture maps to hardware-parallel implementations:

Table 8: Mapping of adaptive training components to FPGA hardware resources.

Component	FPGA Implementation
Frozen backbone	RTL conv pipelines or soft-core with HLS
Mutable heads	Block RAM or HBM-backed weight arrays
Gradient engine	Streaming datapath with fixed-point arithmetic
Weight storage	Block RAM dual-port for stable/candidate slots
Gradient buffers	Streaming datapath registers (no external memory)
NVS persistence	SD card or SPI flash controller IP
Temperature sensor	XADC (Xilinx) or on-die thermal diode
Resource guard	Dedicated FSM monitoring utilization counters

FPGA implementations enable **true hardware-parallel** resource gating (all five checks evaluated in a single clock cycle), **pipelined gradient computation** (forward and backward pass stages operating concurrently), and **deterministic timing guarantees** that eliminate the need for wall-clock budgets. The dual-slot model store maps naturally to dual-port Block RAM with single-cycle atomic weight swapping.

4.11.4 Why Full-Model Training Is Infeasible on These Platforms

Conventional neural network training (full backpropagation through all layers) is infeasible on MCU, SoC, and FPGA platforms for three reasons that the present invention directly addresses:

- (1) **Memory:** Full backpropagation requires storing all intermediate activations for the backward pass. For the 112,552-parameter CNN+SE+Transformer model in this invention, full-model training would require storing activations for three convolutional layers, SE attention, projection with LayerNorm, and two transformer encoder layers (self-attention scores, feed-forward intermediates, layer norms), compared to only 694 bytes when training only the classification heads (a single 64-dimensional feature vector plus three small hidden-layer caches: 256 bytes for the projected feature vector, 384 bytes for the three head hidden-layer

activations, and 54 bytes for the three head outputs). The frozen-backbone approach reduces activation storage by approximately $715\times$ ($496,494 / 694 \approx 715$), making it feasible within the PSRAM budget of a sub-\$10 MCU.

- (2) **Compute:** Backpropagation through convolutional layers requires transposed convolution operations and per-element gradient accumulation across spatial dimensions. On a 240 MHz dual-core MCU without matrix-multiply acceleration, full-backbone gradient computation would take several seconds per training step — consuming a substantial fraction of the 10-second inference window and preventing real-time operation. In contrast, head-only training was measured at 10–24 ms per episode on the ESP32-S3 at 240 MHz, well within the 2,000 ms wall-clock budget.
- (3) **Numerical stability:** Deep gradient chains through many convolutional and transformer layers are prone to vanishing and exploding gradients, especially in single-precision (FP32) arithmetic without the numerical libraries available on GPU platforms. By limiting the gradient path to three fully-connected layers with intermediate LayerNorm, the invention keeps the gradient chain short and numerically stable with simple safeguards (gradient clipping, weight clamping, NaN/Inf detection).

The frozen-backbone, mutable-head design is therefore not merely an optimization — it is the **architectural enabler** that makes on-device training physically possible on hardware that cannot support conventional training methods.

4.11.5 Key Insight: Platform-Independent Innovation

The core inventive contribution is the **algorithmic architecture** — the combination of frozen backbone, mutable heads, multi-trigger feedback, five-gate resource gating, dual-slot versioning, A/B validation, and safety lockout — not the specific hardware implementation. This architecture is equally implementable on:

- **Microcontrollers (MCU):** Demonstrated on ESP32-S3 (this application)

- **System-on-Chip (SoC):** Applicable to ARM Cortex-M7/A series with DSP/NPU
- **FPGA:** Applicable to Xilinx Zynq, Intel Cyclone, or similar platforms
- **Application-Specific Integrated Circuits (ASIC):** The architecture could be synthesized into dedicated silicon for high-volume medical device production

The patent claims are drafted at the algorithmic level, covering the method and system regardless of the underlying compute platform, while the reference implementation on ESP32-S3 demonstrates practical feasibility on the most resource-constrained end of the spectrum.

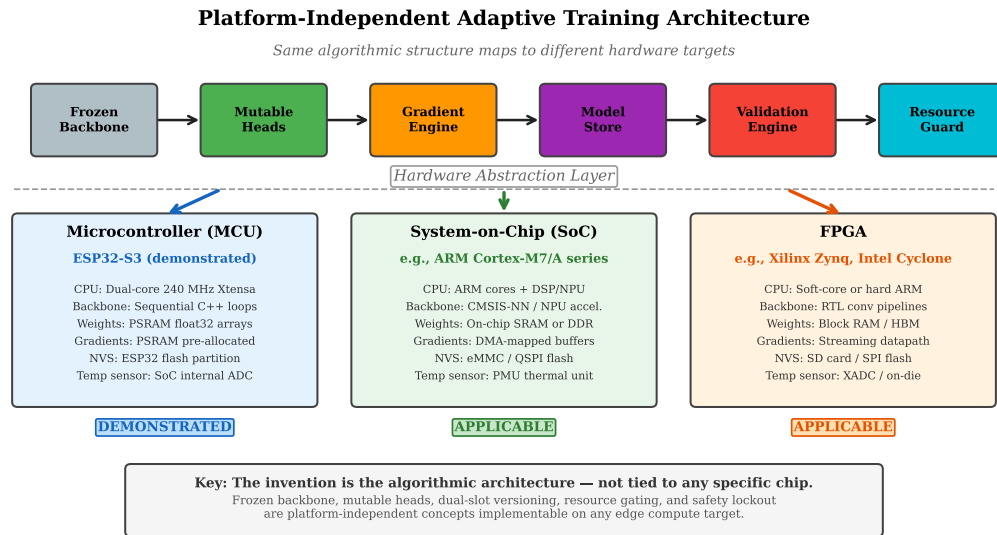


Figure 12: Platform-independent adaptive training architecture. The same algorithmic structure maps to Microcontroller (MCU, demonstrated), System-on-Chip (SoC, applicable), and FPGA (applicable) targets through a Hardware Abstraction Layer.

5. CLAIMS

5.1 Independent Claims

Claim 1. A method for on-device adaptive training of a multi-task neural network on a resource-constrained microcontroller, the method comprising:

- (a) executing a frozen convolutional neural network backbone on the microcontroller to extract feature vectors from multi-sensor biomedical input data, wherein the backbone parameters are fixed in read-only flash memory and are not modified during on-device training;
- (b) performing inference through a plurality of mutable classification heads connected to the backbone output, each head corresponding to a distinct biomedical classification task;
- (c) detecting at least one adaptation trigger, selected from the group consisting of: confidence drift detection based on an exponential moving average of prediction probabilities, receipt of a user-provided ground-truth correction label, and expiration of a periodic adaptation timer;
- (d) verifying, prior to initiating a training episode, that a plurality of hardware resource conditions are simultaneously satisfied, including at least available memory, system-on-chip temperature, and elapsed time since a previous training episode;
- (e) performing backpropagation and stochastic gradient descent with momentum through the mutable classification heads using pre-allocated buffers, without invoking dynamic memory allocation and without employing external machine learning libraries;
- (f) evaluating, entirely locally on the microcontroller and without transmitting feature vectors, correction labels, or model weights to any external server or cloud service, a candidate model resulting from step (e) against a stable model on a held-out validation buffer comprising recent labeled samples;
- (g) conditionally promoting the candidate model to replace the stable model if and only if an accuracy of the candidate model, as determined by the local evaluation of step (f), meets a promotion criterion relative to an accuracy of the stable model for all classification tasks, and otherwise rolling back to the stable model; and
- (h) persisting weights of the promoted stable model to non-volatile flash storage for survival across power cycles.

Claim 2. A system for on-device adaptive neural network training on embedded hardware, the system comprising:

- (a) a microcontroller system-on-chip with internal SRAM and external PSRAM;
- (b) a plurality of biomedical sensors connected to the microcontroller, providing real-time physiological and inertial data;
- (c) a frozen neural network backbone stored in flash memory, configured to extract feature vectors from the sensor data;
- (d) a plurality of mutable classification heads stored in pre-allocated PSRAM buffers, each head configured for a distinct biomedical classification task;
- (e) a gradient engine configured to perform backpropagation and weight updates through the classification heads using pre-allocated gradient, momentum, and activation cache buffers, with no dynamic memory allocation during training;
- (f) a model store maintaining a stable weight slot and a candidate weight slot in pre-allocated PSRAM, configured for atomic promotion and $O(1)$ rollback between the stable weight slot and the candidate weight slot, and for non-volatile flash persistence;
- (g) a feedback controller configured to detect adaptation triggers based on confidence drift, user corrections, and periodic scheduling;
- (h) a resource guard configured to verify hardware resource availability before permitting training episodes;
- (i) a validation engine configured to perform, entirely on the microcontroller system-on-chip and without communicating with any external server, a classification-accuracy-based A/B comparison between a candidate model occupying the candidate weight slot and a stable model occupying the stable weight slot on a held-out validation buffer, wherein the atomic promotion of the model store is conditioned exclusively on a determination by the validation

engine that the candidate model satisfies an accuracy-based promotion criterion relative to the stable model, rather than on a firmware boot-success signal; and

- (j) a safety lockout mechanism configured to disable adaptation after a configurable number of consecutive validation failures.

Claim 3. A non-transitory computer-readable medium storing firmware instructions that, when executed by a microcontroller system-on-chip, cause the system-on-chip to:

- (a) acquire multi-channel biomedical sensor data at a predetermined sampling rate;
- (b) perform multi-task neural network inference using a frozen convolutional backbone and a plurality of mutable classification heads;
- (c) monitor prediction confidence using an exponential moving average to detect distributional drift;
- (d) receive and buffer user-provided correction labels paired with corresponding feature vectors;
- (e) conditionally execute a bounded training episode when adaptation triggers are detected and hardware resource constraints are satisfied, the training episode comprising backpropagation through the classification heads using pre-allocated buffers;
- (f) validate candidate models against a stable model using A/B accuracy comparison, the comparison being performed entirely on the system-on-chip without communicating with any external server or cloud service;
- (g) promote validated candidates through atomic weight replacement, conditioned on the accuracy-based validation of step (f) rather than a firmware boot-success signal, with non-volatile flash persistence; and

- (h) enforce, in combination, a plurality of safety constraints including gradient clipping, weight magnitude clamping, NaN/Inf detection, weight divergence scanning, and automatic lock-out after consecutive validation failures, the combination of constraints collectively operating within a zero-dynamic-allocation, externally-library-free training loop to guard against degradation of on-device classification performance.

5.2 Dependent Claims

Claim 4. The method of Claim 1, wherein the hardware resource conditions of step (d) comprise at least five simultaneous checks: available PSRAM exceeding a minimum threshold, available heap memory exceeding a minimum threshold, system-on-chip temperature below a maximum threshold, elapsed time since a previous training episode exceeding a minimum cooldown interval, and most recent inference latency below a maximum threshold.

Claim 5. The method of Claim 1, wherein step (e) further comprises gradient clipping, comprising computing an L_2 norm of a complete gradient vector across all trainable parameters in a classification head, and uniformly scaling all gradients when the norm exceeds a configurable threshold.

Claim 6. The method of Claim 1, wherein step (e) further comprises clamping every updated weight to a configurable magnitude bound, and aborting the training step with weight rollback if any NaN or Inf value is detected in the forward pass, backward pass, or updated weights.

Claim 7. The method of Claim 1, wherein the exponential moving average of step (c) is computed independently for each classification task head, and the adaptation trigger fires when the EMA drops below a configurable threshold for a configurable number of consecutive inference windows.

Claim 8. The method of Claim 1, wherein step (e) operates within a wall-clock budget, and training terminates early if the budget is exceeded, using whatever gradient steps have been completed.

Claim 9. The method of Claim 1, further comprising maintaining a safety lockout counter that is incremented when a candidate model fails validation and reset to zero when a candidate is

successfully promoted, wherein adaptation is disabled when the counter reaches a configurable maximum.

Claim 10. The method of Claim 1, further comprising a factory reset operation that restores original model weights from read-only flash memory, clears all adapted weights from non-volatile storage, and resets the adaptation generation counter.

Claim 11. The system of Claim 2, wherein the gradient engine implements cross-entropy loss with a softmax output layer, and wherein the gradient with respect to logits z_i is computed as $\hat{y}_i - \mathbf{1}[i = c]$ (where c is the index of the ground-truth class label, $\hat{y}_i$ is the predicted probability for class i , and $\mathbf{1}[i = c]$ is the indicator function that equals 1 when $i = c$ and 0 otherwise), enabling a single closed-form gradient computation for the output layer.

Claim 12. The system of Claim 2, wherein the model store serializes all classification head weights into a single contiguous binary blob for non-volatile flash storage, and maintains a monotonically increasing generation counter that is persisted alongside the weights.

Claim 13. The system of Claim 2, wherein the validation engine performs A/B comparison by: (a) saving candidate weights to the candidate slot, (b) evaluating candidate accuracy, (c) loading stable weights from the stable slot, (d) evaluating stable accuracy on the same validation data, and (e) restoring candidate weights from the candidate slot, all without dynamic memory allocation.

Claim 14. The system of Claim 2, wherein the resource guard reads a system-on-chip internal temperature sensor using API-adaptive code that selects between different hardware abstraction layer interfaces at compile time based on the SDK version.

Claim 15. The method of Claim 1, wherein the multi-task neural network simultaneously classifies at least three distinct biomedical tasks from a shared feature representation, and wherein step (g) requires the promotion criterion to be independently satisfied for every one of the at least three distinct biomedical tasks before promoting the candidate model.

Claim 16. The system of Claim 2, wherein the feedback controller combines at least three independent trigger mechanisms: an unsupervised confidence drift detector that requires no labeled data, a supervised user correction interface that buffers labeled samples in a ring buffer, and a periodic timer that triggers adaptation at configurable intervals.

Claim 17. The method of Claim 1, wherein the microcontroller is an ESP32-S3 system-on-chip operating at 240 MHz with 8 MB PSRAM and 8 MB flash, the biomedical input data comprises electrocardiogram (ECG), photoplethysmogram (PPG), and three-axis accelerometer signals sampled at 100 Hz, and the total firmware overhead for the adaptive training system is less than 10 KB of flash.

Claim 18. The system of Claim 2, wherein all training buffers including gradient buffers, momentum buffers, activation caches, model store slots, correction buffers, and validation buffers are pre-allocated during system initialization, and wherein zero dynamic memory allocation calls occur during any training episode.

Claim 19. The system of Claim 2, wherein the microcontroller system-on-chip is selected from the group consisting of: a microcontroller unit (MCU), a system-on-chip (SoC) comprising ARM cores with DSP or NPU co-processors, and a field-programmable gate array (FPGA), and wherein the frozen backbone, mutable heads, gradient engine, model store, validation engine, and resource guard interact through a hardware abstraction layer that maps to platform-specific resources including addressable memory for weight storage, arithmetic units for gradient computation, persistent storage for non-volatile weight serialization, and thermal sensors for temperature gating.

Claim 20. The method of Claim 1, wherein the method is implemented on a field-programmable gate array (FPGA) and the resource gating of step (d) is performed by a dedicated finite state machine that evaluates all hardware constraints in a single clock cycle, and wherein the dual-slot model versioning uses dual-port block RAM for single-cycle atomic weight swapping between stable and candidate slots.

6. ABSTRACT

A system and method, referred to herein as “EdgeGuard,” for enabling neural network learning on resource-constrained edge hardware (microcontrollers, systems-on-chip, and field-programmable gate arrays) without a GPU, cloud connectivity, or external machine learning frameworks. A CNN-SE-Transformer backbone is frozen in read-only memory while mutable classification heads are adapted on-device through feedback-driven backpropagation using zero-allocation pre-allocated buffers. Hardware-enforced resource gating conditions training on simultaneous memory, thermal, timing, and CPU constraints. A dual-slot model versioning system with non-volatile flash persistence enables atomic promotion and rollback, gated by local accuracy-based validation against a stable model, with safety lockout after consecutive validation failures. Nine concentric safety mechanisms prevent on-device adaptation from degrading performance below baseline. The platform-independent architecture maps to microcontroller, system-on-chip, and FPGA targets via a hardware abstraction layer. Demonstrated on ESP32-S3 hardware with a sub-\$10 bill-of-materials, processing five biomedical sensor channels at 100 Hz, operating autonomously with no internet or cloud dependency.

7. DRAWINGS

The following drawings are incorporated by reference and form part of this application:

- **Figure 1:** On-device adaptive training system architecture showing the frozen CNN+SE+Transformer backbone, mutable classification heads, and seven integrated components.
- **Figure 2:** End-to-end data flow from sensor acquisition through CNN inference, feedback detection, gradient computation, validation, and model promotion.
- **Figure 3:** Decision flowchart for the complete on-device adaptation cycle from inference through trigger evaluation, resource gating, training, validation, and promotion/rollback.

- **Figure 4:** Parameter partitioning between the frozen CNN+SE+Transformer backbone (107,568 parameters) and the trainable classification heads (4,984 parameters) across three task-specific heads.
- **Figure 5:** Gradient engine dataflow showing forward pass, backward pass with cached activations, gradient clipping, and SGD with momentum update.
- **Figure 6:** Dual-slot model versioning architecture showing stable slot, candidate slot, promotion path, rollback path, NVS flash persistence, and factory reset.
- **Figure 7:** NVS flash persistence lifecycle showing factory weight loading, on-device training, validation, persistence to NVS flash, and weight reload after power cycling.
- **Figure 8:** Feedback controller state machine with four states (Monitoring, Triggered, Training, Locked Out) and three independent trigger paths.
- **Figure 9:** Hardware-enforced resource gating decision tree showing five independent checks that must all pass simultaneously before training is permitted.
- **Figure 10:** A/B validation protocol (zero-allocation) showing the 5-step procedure for evaluating candidate and stable models using pre-allocated weight slots.
- **Figure 11:** Nine concentric safety layers surrounding the deployed model, from innermost (NaN/Inf guards) to outermost (factory reset).
- **Figure 12:** Platform-independent adaptive training architecture showing how the same algorithmic structure maps to Microcontroller (MCU, demonstrated), System-on-Chip (SoC, applicable), and FPGA (applicable) targets.